

Kiln: Run and Deploy Guide

From zero to a live app on Render and Neon, with an optional local Ollama model. Updated 8 October 2026.

What you are setting up

Kiln runs as three pieces:

1. **Neon** stores the data (a free Postgres database).
2. **Render** runs the app: the `kiln` service (website and API) and the `kiln-ai` service (predictions and the code sandbox).
3. **Ollama** (optional) runs the small language model on your own PC for the tutor's written answers.

Your old MongoDB is not used by Kiln. Nothing in Kiln touches it, and you can leave it installed.

What you need before you start

- The `kiln.zip` file, unzipped on your computer.
- A free account on each of: **GitHub**, **Neon** (neon.com), **Render** (render.com).
- For the optional model: **Ollama** and a free **ngrok** account.

Part 1: Create the database on Neon

1. Go to **neon.com** and sign up.
2. Click **Create project**. Choose the **Singapore** region (it matches Render's Singapore region).
3. Click the **Connect** button.
4. Turn **Connection pooling** on.
5. Copy the long line that starts with `postgresql://` and paste it into Notepad. You need it in Part 3.
6. If the line ends with `&channel_binding=require`, delete just that part. Keep `?sslmode=require`.

Part 2: Put the project on GitHub

GitHub's website accepts only about 100 files per upload and this project has about 197, so use GitHub Desktop.

1. Install **GitHub Desktop** from desktop.github.com and sign in.
2. Click **File > New repository**. Name it `Kiln` and click **Create repository**.
3. Open the unzipped project folder, select everything inside it (use Select All so hidden items like `.github` come along), and copy it into the new `Kiln` folder that GitHub Desktop created.
4. Return to GitHub Desktop. You will see a long list of files.
5. Type `Kiln` in the Summary box and click **Commit to main**.
6. Click **Publish repository**.
7. Open the repository on github.com and check that `render.yaml` and the folders `client`, `server` and `ai-service` are on the very first page. If you only see one folder called `kiln`, the files went in one level too deep: redo step 3.

The file named `.env` in the list is really `.env.example`. It contains no secrets, so it is safe to upload. Never upload a real `.env` file.

Part 3: Deploy on Render

1. Go to render.com and sign up with GitHub.
2. Click **New**, then **Blueprint**.
3. Choose your `Kiln` repository.
4. Render shows two services, `kiln-ai` and `kiln`, and one box: `DATABASE_URL`.
5. Paste the Neon line from Part 1 into that box.
6. Click **Deploy Blueprint**.
7. Wait 5 to 10 minutes. Both services must show **Live** in green.

How to check the deploy worked

1. Click `kiln` and open the **Logs** tab. Near the bottom you should see:

```
Seeded 3 courses, 9 lessons ... 120 simulated learners.
Kiln API on :10000
```

2. Click the web address at the top of the `kiln` page (it looks like `https://kiln-xxxx.onrender.com`). The first load can take up to a minute.
3. Add `/api/health` to the end of the address. You should see `"status":"ok"`.
4. On the sign-in page click **Student demo**, then try the dashboard, a lesson, a quiz and a coding exercise.
5. Click **Teacher demo** to see the dashboard. If predictions show, the two Render services are talking to each other.

Demo accounts, password `Demo@1234`: `student@kiln.dev` and `teacher@kiln.dev`.

At this point the app is online and in **demo mode**: the tutor shows matching lesson text and quizzes come from the built-in question bank. Everything else works fully.

Part 4 (optional): Turn on the language model with Ollama

The llama model runs on your own PC, so Render reaches it through a tunnel called ngrok. This works only while your PC is on and awake, Ollama is running, and the ngrok window is open.

A. Check Ollama

Open PowerShell and run:

```
ollama list
```

You should see `llama3.2:1b`. If not, run `ollama pull llama3.2:1b`. Then open `http://localhost:11434` in your browser. It should say **Ollama is running**.

B. Install ngrok and add your token

1. Sign up at **ngrok.com** (free).
2. In PowerShell run:

```
winget install ngrok.ngrok
```

3. Close PowerShell and open a new window.
4. In the ngrok dashboard, open **Your Authtoken** and copy it.
5. Run this with your real token, once:

```
ngrok config add-authtoken YOUR_TOKEN
```

C. Open the tunnel

```
ngrok http 11434 --host-header="localhost:11434"
```

Copy the `https://....ngrok-free.app` address from the **Forwarding** line and keep this window open. The `--host-header` part is required.

D. Tell Render about it

1. In Render, click the **kiln** service, then **Environment** in the left menu.
2. Click **Edit**, then add three variables:

Key	Value
LLM_PROVIDER	ollama
OLLAMA_URL	your ngrok address, with no <code>/</code> at the end
OLLAMA_MODEL	llama3.2:1b

3. Leave the seven existing variables alone.
4. Click **Save, rebuild, and deploy** and wait a minute or two.

E. Check it works

1. Open `your-kiln-address/api/health`. You should see `"llm":"live"` and `"provider":"ollama"`.
2. In the app, open **Ask the tutor** and ask "What is a Python list?". The first answer can take a minute while the model loads.

Using the app day to day

- The Render link works from any device and any network.
- The free **kiln** service sleeps after about 15 minutes idle, so the next visit takes around 50 seconds to wake.
- Sign-in, courses, lessons, quizzes, coding exercises and the teacher dashboard never need your PC.
- The tutor's written answers need your PC, Ollama and ngrok all running. If any is off, the tutor says the model is unavailable and the rest of the app keeps working.

- Each time you close and reopen ngrok the address changes. Update `OLLAMA_URL` in Render's Environment page and save. A fixed free ngrok domain avoids this: in the ngrok dashboard open **Domains**, create one, and start the tunnel with `--url=YOUR-DOMAIN`.
- Everyone who uses your link shares your PC's model. Keep the link private if you do not want others using your computer.

Optional: run it on your own computer instead

Use this only if you want to develop or test locally. Kiln needs Postgres, so use your Neon string or install Postgres.

1. Copy `.env.example` to `.env` and set `DATABASE_URL`,
`AI_SERVICE_URL=http://127.0.0.1:8000` and a long random `JWT_SECRET`.

2. Window 1:

```
cd kiln\ai-service
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
uvicorn app.main:app --port 8000
```

3. Window 2 (run `npm run seed` only once, because it erases all data):

```
cd kiln\server
npm install
npm run seed
npm run dev
```

4. Window 3:

```
cd kiln\client
npm install
npm run dev
```

5. Open `http://localhost:5173`.

On later runs only activate the venv and start the three services.

Troubleshooting

What you see	What to do
Cannot reach the database in the kiln logs	Re-paste <code>DATABASE_URL</code> in the kiln Environment page. Remove <code>&channel_binding=require</code> if present.
Render cannot find the blueprint	<code>render.yaml</code> is not on the first page of the repository. Redo Part 2, step 3.
Teacher dashboard says it is using rules, not the model	The private network address may not work on the free plan. On kiln, set <code>AI_SERVICE_URL</code> to the public address of kiln-ai (it looks like <code>https://kiln-ai-xxxx.onrender.com</code>) and save.
kiln-ai restarts with an out-of-memory message	Move only kiln-ai to the Starter plan.
Health page still says <code>"llm": "demo"</code>	Check the three Ollama variables are spelled exactly as shown, then save and wait for the redeploy.
Tutor says the model is unavailable	Check that your PC is awake, Ollama is running, and the ngrok window is open. Open the ngrok address in a browser: it should say Ollama is running. If the address changed, update <code>OLLAMA_URL</code> .
Everything is slow after a pause	That is the free plan waking up. Wait about a minute.

If something still fails, copy the first line containing the word error from the **Logs** tab of the failing service and ask for help with that line.

Safety notes

- Never upload a real `.env` file to GitHub or share it. If one was ever shared, create new keys.
- If you add a paid model key, set a spending cap on it.

- The AI service has a public address on Render. The code sandbox is protected by a shared secret that Render already set. For a real public launch, change `kiln-ai` to a private service.